Lab 3: Gesture Recognition using Convolutional Neural Networks

In this lab you will train a convolutional neural network to make classifications on different hand gestures. By the end of the lab, you should be able to:

1. Load and split data for training, validation and testing
2. Train a Convolutional Neural Network
3. Apply transfer learning to improve your model

Note that for this lab we will not be providing you with any starter code. You should be able to take the code used in previous labs, tutorials and lectures and modify it accordingly to complete the tasks outlined below.

What to submit

Submit a PDF file containing all your code, outputs, and write-up from parts 1-5. You can produce a PDF of your Google Colab file by going to **File > Print** and then save as PDF. The Colab instructions has more information. Make sure to review the PDF submission to ensure that your answers are easy to read. Make sure that your text is not cut off at the margins.

Do not submit any other files produced by your code.

Include a link to your colab file in your submission.

Please use Google Colab to complete this assignment. If you want to use Jupyter Notebook, please complete the assignment and upload your Jupyter Notebook file to Google Colab for submission.

Colab Link

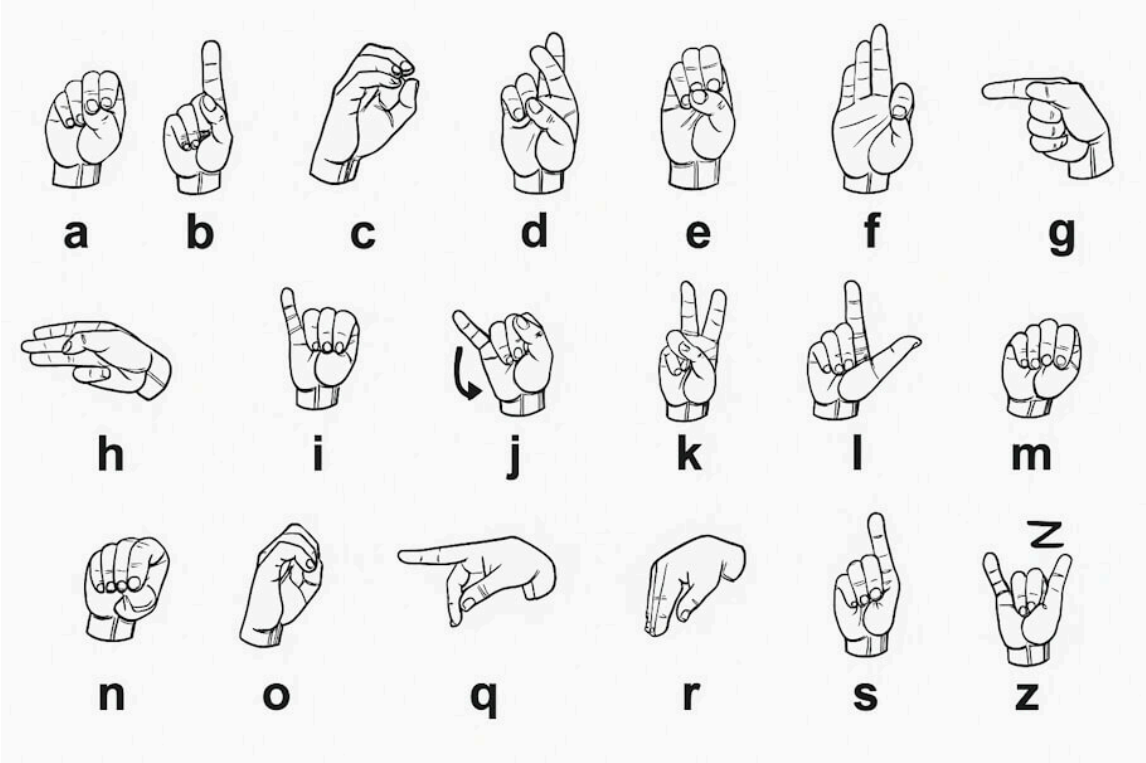
Include a link to your colab file here

Colab Link:
https://colab.research.google.com/github/sissi820li/APS360/blob/main/Lab%203/Lab%203%20G

Dataset

American Sign Language (ASL) is a complete, complex language that employs signs made by moving the hands combined with facial expressions and postures of the body. It is the primary language of many North Americans who are deaf and is one of several

communication options used by people who are deaf or hard-of-hearing. The hand gestures representing English alphabet are shown below. This lab focuses on classifying a subset of these hand gesture images using convolutional neural networks. Specifically, given an image of a hand showing one of the letters A-I, we want to detect which letter is being represented.



Part B. Building a CNN [50 pt]

For this lab, we are not going to give you any starter code. You will be writing a convolutional neural network from scratch. You are welcome to use any code from previous labs, lectures and tutorials. You should also write your own code.

You may use the PyTorch documentation freely. You might also find online tutorials helpful. However, all code that you submit must be your own.

Make sure that your code is vectorized, and does not contain obvious inefficiencies (for example, unecessary for loops, or unnecessary calls to unsqueeze()). Ensure enough comments are included in the code so that your TA can understand what you are doing. It is your responsibility to show that you understand what you write.

This is much more challenging and time-consuming than the previous labs. Make sure that you give yourself plenty of time by starting early.

1. Data Loading and Splitting [5 pt]

Download the anonymized data provided on Quercus. To allow you to get a heads start on this project we will provide you with sample data from previous years. Split the data into training, validation, and test sets.

Note: Data splitting is not as trivial in this lab. We want our test set to closely resemble the setting in which our model will be used. In particular, our test set should contain hands that are never seen in training!

Explain how you split the data, either by describing what you did, or by showing the code that you used. Justify your choice of splitting strategy. How many training, validation, and test images do you have?

For loading the data, you can use plt.imread as in Lab 1, or any other method that you choose. You may find torchvision.datasets.ImageFolder helpful. (see https://pytorch.org/docs/stable/torchvision/datasets.html?highlight=image%20folder#torchvision.datasets.ImageFolder)

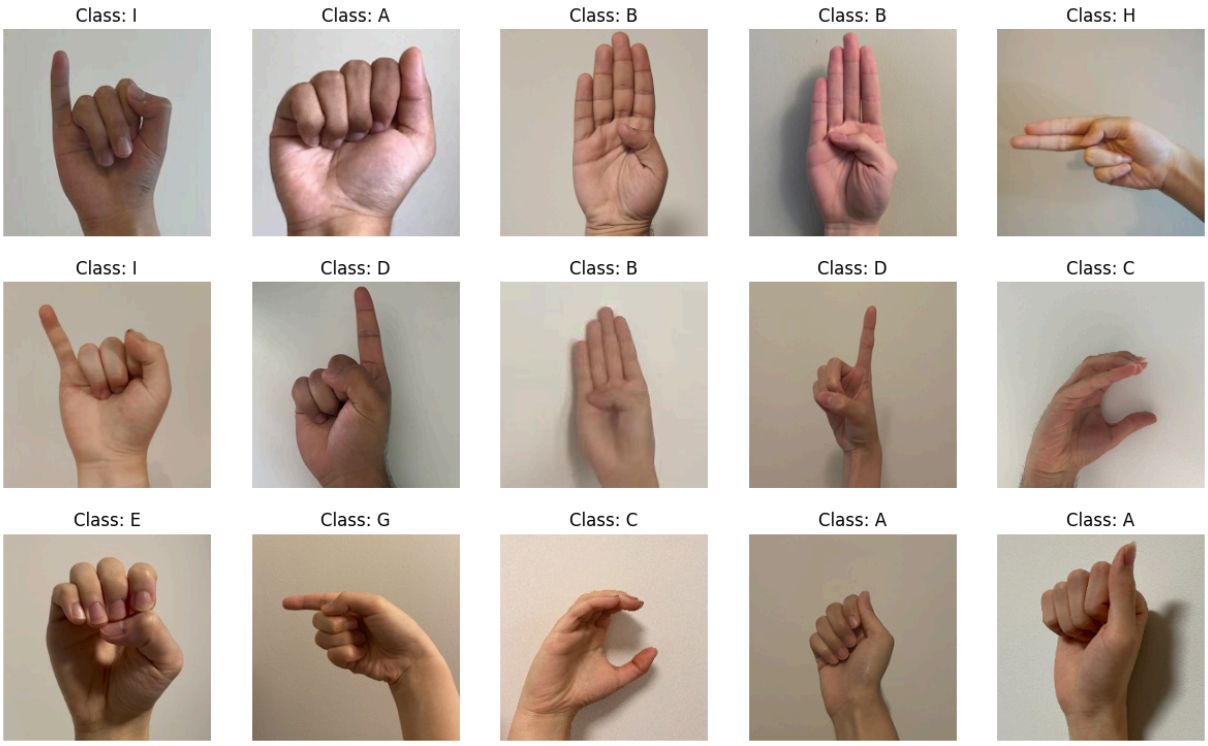
In [1]:
```
import numpy as np
import time
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
from torch.utils.data.sampler import SubsetRandomSampler
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import torchvision.transforms.functional as TF
```

In [2]:
```
# Load data
def get_relevant_indices(dataset, classes, target_classes):
    # Return the indices for datapoints in the dataset that belongs to the desired
    indices = []
    for i in range(len(dataset)): # Check if the label is in the target classes
        label_index = dataset[i][1]
        label_class = classes[label_index]
        if label_class in target_classes:
            indices.append(i)
    return indices

def get_data_loader(target_classes, batch_size):
    # Splits the data into training, validation and testing datasets. Returns data
    classes = ('A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I')

    transform = transforms.Compose([transforms.Resize((224, 224)),
                transforms.ToTensor()]) # 224x224 pixels

    data_path = "C:\\Users\\sissi\\OneDrive\\Desktop\\Code\\APS360\\APS360\\Lab 3\\
    trainset = torchvision.datasets.ImageFolder(data_path, transform = transform)

    relevant_indices = get_relevant_indices(trainset, classes, target_classes)
    np.random.seed(1000) # Keeps the random shuffle
```

```
    np.random.shuffle(relevant_indices)

    # Calculate the cut-off points for a 70%/15%/15% split of the data
    split1 = int(len(relevant_indices) * 0.70)
    split2 = int(len(relevant_indices) * 0.85)

    train_indices = relevant_indices[:split1] # 0% to 70% of data
    val_indices = relevant_indices[split1:split2] # 70% to 85%
    test_indices = relevant_indices[split2:] # 85% to 100%

   # Create samplers for all three lists
    train_sampler = SubsetRandomSampler(train_indices)
    val_sampler = SubsetRandomSampler(val_indices)
    test_sampler = SubsetRandomSampler(test_indices)

    # Load data. The sampler ensures they only pull their assigned chunk
    train_loader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, num
    val_loader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, num_w
    test_loader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, num_
    return train_loader, val_loader, test_loader, classes

train_loader, val_loader, test_loader, classes = get_data_loader(
    target_classes=["A", "B", "C", "D", "E", "F", "G", "H", "I"],
    batch_size=1)
```

In [3]:

```
# Visualize data
plt.figure(figsize=(15, 9)) # Set up the canvas size for 3x5 grid

k = 0
for images, labels in train_loader:
    image = images[0] # Batch size is 1 so take the first image
    label_idx = labels[0].item()

    img = np.transpose(image.numpy(), [1, 2, 0])  # Place the colour channel at the

    # Plot with 3 rows and 5 columns
    plt.subplot(3, 5, k+1)
    plt.axis('off')

    # Show the letter above the image
    plt.title(f"Class: {classes[label_idx]}")
    plt.imshow(img)

    k += 1
    if k > 14: # Break the loop after 15 images
        break

plt.show()
```



In [4]:
```
total_data = len(train_loader) + len(val_loader) + len(test_loader)

print('There are', len(train_loader), 'training images, which is {:.2f} % of the en
print('There are', len(val_loader), 'validation images, which is {:.2f} % of the en
print('There are', len(test_loader), 'test images, which is {:.2f} % of the entire
```

```
There are 1553 training images, which is 69.99 % of the entire dataset.
There are 333 validation images, which is 15.01 % of the entire dataset.
There are 333 test images, which is 15.01 % of the entire dataset.
```

2. Model Building and Sanity Checking [15 pt]

Part (a) Convolutional Network - 5 pt

Build a convolutional neural network model that takes the (224x224 RGB) image as input, and predicts the gesture letter. Your model should be a subclass of nn.Module. Explain your choice of neural network architecture: how many layers did you choose? What types of layers did you use? Were they fully-connected or convolutional? What about other decisions like pooling layers, activation functions, number of channels / hidden units?

Output = (W - F + 2P)/S + 1

W = input width/height, F = filter size, P = padding, S = stride

conv1: (224 - 5 + 0)/1 + 1 = 220

pool: 220/2 = 110

conv2: (110 - 5 + 0)/1 + 1 = 106

pool: 106/2 = 53

The above calculations helped me determine the number of channels and hidden units. The output of the convolution layers are 10 channels of 53x53 images. I chose a CNN architecture consisting of 2 convolutional layers, 2 pooling layers, and 2 fully connected layers. The two convolutional layers allow the model to capture both low-level edges and more complex features of the hand gestures. Max pooling is applied to reduce the spatial dimensions of the feature maps which will reduce the number of learnable parameters. This allows the model to train faster. Finally, the extracted features are flattened and passed to the 2 fully-connected layers for the final classification. The ReLU activation function is utilized to introduce non-linearity into the network.

In [4]:
```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.name = "cnn"

        # Convolution layers
        self.conv1 = nn.Conv2d(3, 5, 5) # (in_channels, out_channels, kernel_size)
        self.pool = nn.MaxPool2d(2, 2) # (kernel_size, stride)
        self.conv2 = nn.Conv2d(5, 10, 5) # in_channels match out_channels of conv1

        # Fully connected layers
        self.fc1 = nn.Linear(10 * 53 * 53, 32) # (in_features, out_features), 32 hi
        self.fc2 = nn.Linear(32, 9) # in_features matches out_features of fc1, out_

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 10 * 53 * 53) # Flatten 3D tensor into a 1D line of 28,090 n
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        # x = x.squeeze(1) # Flatten to batch_size

        return x

cnn = CNN()
```

Part (b) Training Code - 5 pt

Write code that trains your neural network given some training data. Your training code should make it easy to tweak the usual hyperparameters, like batch size, learning rate, and the model object itself. Make sure that you are checkpointing your models from time to time (the frequency is up to you). Explain your choice of loss function and optimizer.

In [5]:
```
# Obtain accuracy
def get_accuracy(model, batch_size, train=False):
    if train:
        data = train_loader
    else:
```

```
        data = val_loader

    correct = 0
    total = 0
    for imgs, labels in torch.utils.data.DataLoader(data, batch_size=batch_size):
        output = model(imgs)
        # Select index with maximum prediction score
        pred = output.max(1, keepdim=True)[1]
        correct += pred.eq(labels.view_as(pred)).sum().item()
        total += imgs.shape[0]
    return correct / total
```

For the loss function, I used Cross Entropy instead of Binary Classfication since Cross Entropy is good for many classes, especially when the model needs to confidently choose one correct category out of multiple mutually exclusive options.

I chose the optimizer SGD with momentum 0.9 since it will complete the training faster compared to other options.

In [6]:
```
#  Train the model
def get_model_name(name, batch_size, learning_rate, epoch):
    path = "model_{0}_bs{1}_lr{2}_epoch{3}".format(name,
                                                   batch_size,
                                                   learning_rate,
                                                   epoch)
    return path

def evaluate(net, loader, criterion):
    # Evaluate the network on the validation set
    total_loss = 0.0
    total_err = 0.0
    total_epoch = 0
    for i, data in enumerate(loader, 0):
        inputs, labels = data
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        _, predicted = torch.max(outputs.data, 1)
        corr = predicted != labels
        total_err += int(corr.sum())
        total_loss += loss.item()
        total_epoch += len(labels)
    err = float(total_err) / total_epoch
    loss = float(total_loss) / (i + 1)
    return err, loss

def train_net(net, train_loader, val_loader, batch_size=64, learning_rate=0.01, num
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(net.parameters(), lr=learning_rate, momentum=0.9)

    train_err = np.zeros(num_epochs)
    train_loss = np.zeros(num_epochs)
    val_err = np.zeros(num_epochs)
    val_loss = np.zeros(num_epochs)
```

```
    # Train the network: Loop over the data iterator and sample a new batch of trai
    start_time = time.time()
    for epoch in range(num_epochs): # Loop over the dataset multiple times
        total_train_loss = 0.0
        total_train_err = 0.0
        total_epoch = 0
        for i, data in enumerate(train_loader, 0):
            # Get the inputs
            inputs, labels = data
            # Zero the parameter gradients
            optimizer.zero_grad()
            # Forward pass, backward pass, and optimize
            outputs = net(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            # Calculate the statistics
            _, predicted = torch.max(outputs.data, 1)
            corr = predicted != labels
            total_train_err += int(corr.sum())
            total_train_loss += loss.item()
            total_epoch += len(labels)
        train_err[epoch] = float(total_train_err) / total_epoch
        train_loss[epoch] = float(total_train_loss) / (i+1)
        val_err[epoch], val_loss[epoch] = evaluate(net, val_loader, criterion)
        print(("Epoch {}: Train err: {}, Train loss: {} |"+
               "Validation err: {}, Validation loss: {} |").format(
                   epoch + 1,
                   train_err[epoch],
                   train_loss[epoch],
                   val_err[epoch],
                   val_loss[epoch]))
        # Save the current model (checkpoint) to a file
        model_path = get_model_name(net.name, batch_size, learning_rate, epoch)
        torch.save(net.state_dict(), model_path)
    print('Finished Training')
    end_time = time.time()
    elapsed_time = end_time - start_time
    print("Total time elapsed: {:.2f} seconds".format(elapsed_time))
    # Write the train/test loss/err into CSV file for plotting later
    epochs = np.arange(1, num_epochs + 1)
    np.savetxt("{}_train_err.csv".format(model_path), train_err)
    np.savetxt("{}_train_loss.csv".format(model_path), train_loss)
    np.savetxt("{}_val_err.csv".format(model_path), val_err)
    np.savetxt("{}_val_loss.csv".format(model_path), val_loss)
```

In [7]:

```
def plot_training_curve(path):
    """ Plots the training curve for a model run, given the csv files
    containing the train/validation error/loss.

    Args:
        path: The base path of the csv files produced during training
    """
    import matplotlib.pyplot as plt
    train_err = np.loadtxt("{}_train_err.csv".format(path))
    val_err = np.loadtxt("{}_val_err.csv".format(path))
```

```
    train_loss = np.loadtxt("{}_train_loss.csv".format(path))
    val_loss = np.loadtxt("{}_val_loss.csv".format(path))
    plt.title("Train vs Validation Error")
    n = len(train_err) # number of epochs
    plt.plot(range(1,n+1), train_err, label="Train")
    plt.plot(range(1,n+1), val_err, label="Validation")
    plt.xlabel("Epoch")
    plt.ylabel("Error")
    plt.legend(loc='best')
    plt.show()
    plt.title("Train vs Validation Loss")
    plt.plot(range(1,n+1), train_loss, label="Train")
    plt.plot(range(1,n+1), val_loss, label="Validation")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend(loc='best')
    plt.show()
```

Part (c) “Overfit” to a Small Dataset - 5 pt

One way to sanity check our neural network model and training code is to check whether the model is capable of “overfitting” or “memorizing” a small dataset. A properly constructed CNN with correct training code should be able to memorize the answers to a small number of images quickly.

Construct a small dataset (e.g. just the images that you have collected). Then show that your model and training code is capable of memorizing the labels of this small data set.

With a large batch size (e.g. the entire small dataset) and learning rate that is not too high, You should be able to obtain a 100% training accuracy on that small dataset relatively quickly (within 200 iterations).

In [29]:
```
import torch.utils.data as data

# Create a tiny loader with 64 images
dataiter = iter(train_loader)
images, labels = next(dataiter)

# Package raw math data into (inputs, labels) pairs
debug_dataset = data.TensorDataset(images, labels)
debug_loader = data.DataLoader(debug_dataset, batch_size=64, shuffle=True)

sanity_model = CNN()
train_net(sanity_model, train_loader=debug_loader, val_loader=val_loader, batch_siz
sanity_path = get_model_name(sanity_model.name, batch_size=64, learning_rate=0.001,
#plot_training_curve(sanity_path)

correct = 0
total = 0
for imgs, labels in torch.utils.data.DataLoader(debug_dataset, batch_size=64):
    output = sanity_model(imgs)
    #select index with maximum prediction score
```

```
    pred = output.max(1, keepdim=True)[1]
    correct += pred.eq(labels.view_as(pred)).sum().item()
    total += imgs.shape[0]
print('Accuracy on batch of 64: ', correct / total)
```

```
Epoch 1: Train err: 1.0, Train loss: 2.2301483154296875 |Validation err: 0.906906906
9069069, Validation loss: 2.206731546390522 |
Epoch 2: Train err: 1.0, Train loss: 2.218644618988037 |Validation err: 0.9069069069
069069, Validation loss: 2.206666842595235 |
Epoch 3: Train err: 1.0, Train loss: 2.2002134323120117 |Validation err: 0.906906906
9069069, Validation loss: 2.206668003185375 |
Epoch 4: Train err: 1.0, Train loss: 2.183011054992676 |Validation err: 0.9069069069
069069, Validation loss: 2.206734464691208 |
Epoch 5: Train err: 1.0, Train loss: 2.165149450302124 |Validation err: 0.9069069069
069069, Validation loss: 2.20693105262321 |
Epoch 6: Train err: 1.0, Train loss: 2.1414849758148193 |Validation err: 0.906906906
9069069, Validation loss: 2.207503924498687 |
Epoch 7: Train err: 1.0, Train loss: 2.1109626293182373 |Validation err: 0.897897897
8978979, Validation loss: 2.2086772295805783 |
Epoch 8: Train err: 1.0, Train loss: 2.0692362785339355 |Validation err: 0.900900900
9009009, Validation loss: 2.2109749514061408 |
Epoch 9: Train err: 0.0, Train loss: 2.0108845233917236 |Validation err: 0.879879879
8798799, Validation loss: 2.215648907082933 |
Epoch 10: Train err: 0.0, Train loss: 1.9277390241622925 |Validation err: 0.87987987
98798799, Validation loss: 2.2256000353409364 |
Epoch 11: Train err: 0.0, Train loss: 1.8072422742843628 |Validation err: 0.87987987
98798799, Validation loss: 2.2479634159678095 |
Epoch 12: Train err: 0.0, Train loss: 1.6305737495422363 |Validation err: 0.87987987
98798799, Validation loss: 2.3015995695068314 |
Epoch 13: Train err: 0.0, Train loss: 1.3717820644378662 |Validation err: 0.87987987
98798799, Validation loss: 2.4395738637841142 |
Epoch 14: Train err: 0.0, Train loss: 1.007284164428711 |Validation err: 0.879879879
8798799, Validation loss: 2.8072168488760254 |
Epoch 15: Train err: 0.0, Train loss: 0.5645366311073303 |Validation err: 0.87987987
98798799, Validation loss: 3.6854110617447904 |
Epoch 16: Train err: 0.0, Train loss: 0.19970549643039703 |Validation err: 0.8798798
798798799, Validation loss: 5.231963157111236 |
Epoch 17: Train err: 0.0, Train loss: 0.042811475694179535 |Validation err: 0.879879
8798798799, Validation loss: 7.2811959193335625 |
Epoch 18: Train err: 0.0, Train loss: 0.0064910524524748325 |Validation err: 0.87987
98798798799, Validation loss: 9.648170287852308 |
Epoch 19: Train err: 0.0, Train loss: 0.0007844470092095435 |Validation err: 0.87987
98798798799, Validation loss: 12.228990006922865 |
Epoch 20: Train err: 0.0, Train loss: 7.998623186722398e-05 |Validation err: 0.87987
98798798799, Validation loss: 14.952795111322027 |
Epoch 21: Train err: 0.0, Train loss: 7.152531907195225e-06 |Validation err: 0.87987
98798798799, Validation loss: 17.759946517041246 |
Epoch 22: Train err: 0.0, Train loss: 5.960462772236497e-07 |Validation err: 0.87987
98798798799, Validation loss: 20.598463912267928 |
Epoch 23: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 23.424056253275715 |
Epoch 24: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 26.20018113053239 |
Epoch 25: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 28.897712684608436 |
Epoch 26: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 31.49432087803746 |
Epoch 27: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 33.973686745216895 |
Epoch 28: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 36.324688152507974 |
```

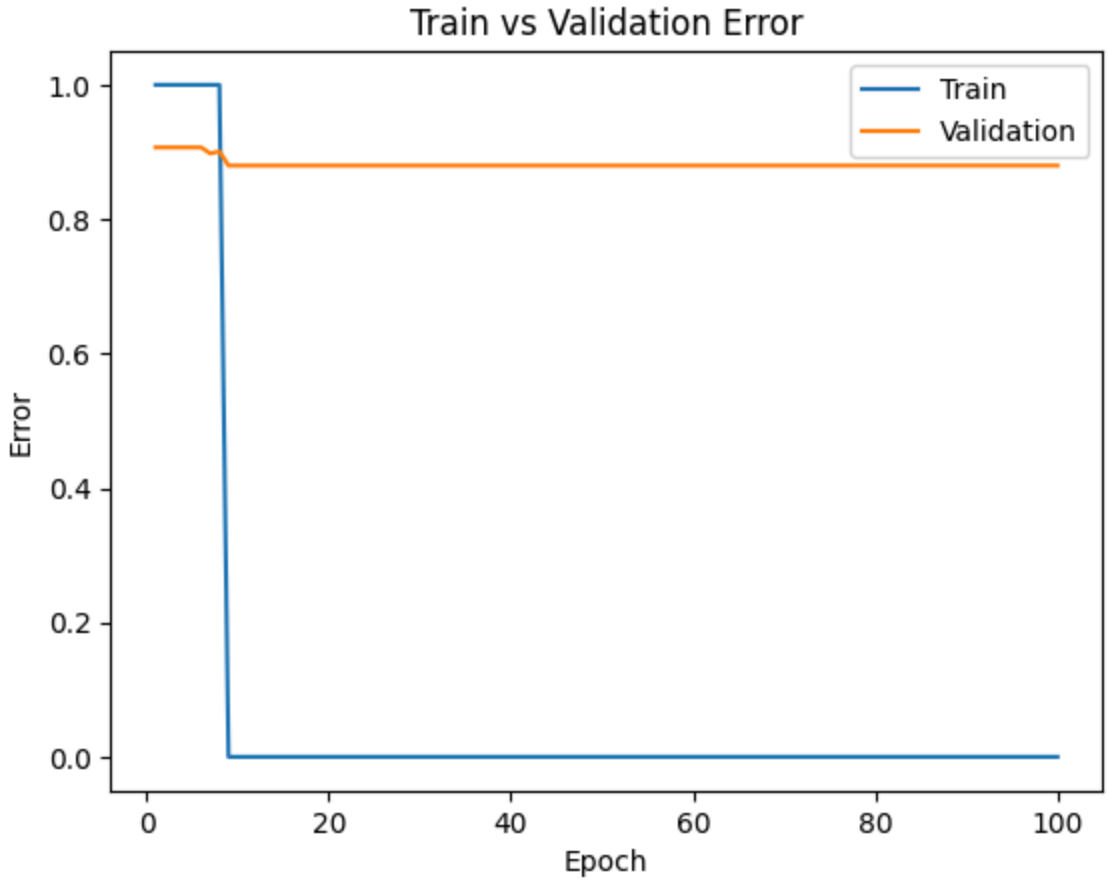
```
Epoch 29: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 38.54055832527779 |
Epoch 30: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 40.618124953261365 |
Epoch 31: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 42.55709845740516 |
Epoch 32: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 44.35944857898059 |
Epoch 33: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 46.02888216127504 |
Epoch 34: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 47.57037991589612 |
Epoch 35: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 48.98981968418614 |
Epoch 36: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 50.293679452157235 |
Epoch 37: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 51.48877661077826 |
Epoch 38: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 52.5820806577757 |
Epoch 39: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 53.58054272763364 |
Epoch 40: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 54.49100273292702 |
Epoch 41: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 55.32008862423825 |
Epoch 42: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 56.074156065244935 |
Epoch 43: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 56.75924690636071 |
Epoch 44: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 57.38107347917986 |
Epoch 45: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 57.9449812284819 |
Epoch 46: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 58.455974361202024 |
Epoch 47: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 58.918689372661234 |
Epoch 48: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 59.33743162412901 |
Epoch 49: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 59.71616568579688 |
Epoch 50: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 60.05854229168133 |
Epoch 51: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 60.36791578642241 |
Epoch 52: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 60.64734504172752 |
Epoch 53: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 60.899651959851695 |
Epoch 54: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 61.12737909952799 |
Epoch 55: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 61.332875294728325 |
Epoch 56: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 61.51825214649464 |
```

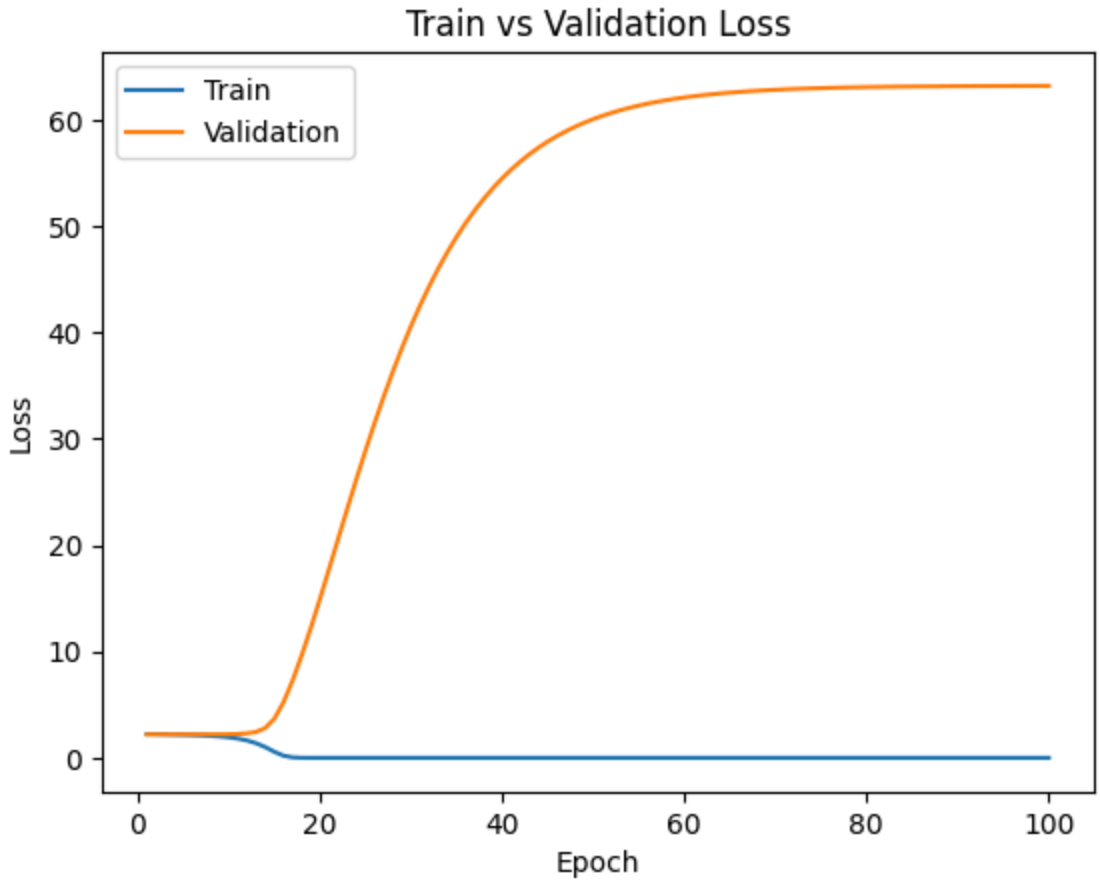
```
Epoch 57: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 61.68544330539646 |
Epoch 58: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 61.83620248972117 |
Epoch 59: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 61.97211503266572 |
Epoch 60: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.09462355421828 |
Epoch 61: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.20503625497446 |
Epoch 62: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.30452846263622 |
Epoch 63: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.3941723706128 |
Epoch 64: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.47493456362246 |
Epoch 65: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.54768134452201 |
Epoch 66: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.61321273437133 |
Epoch 67: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.67223255054371 |
Epoch 68: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.7253838501893 |
Epoch 69: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.77325196595521 |
Epoch 70: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.816349384662985 |
Epoch 71: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.85515835048916 |
Epoch 72: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.89010101180893 |
Epoch 73: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.92156457757807 |
Epoch 74: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.949892336183844 |
Epoch 75: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.9753888333524 |
Epoch 76: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 62.99835063029338 |
Epoch 77: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.01901392893748 |
Epoch 78: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.03761742852472 |
Epoch 79: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.0543669966964 |
Epoch 80: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.0694424970014 |
Epoch 81: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.08301343717375 |
Epoch 82: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.09523137410482 |
Epoch 83: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.10622152838263 |
Epoch 84: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.11612203099706 |
```

```
Epoch 85: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.125028146279824 |
Epoch 86: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.133049125785945 |
Epoch 87: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.140264024247635 |
Epoch 88: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.146763947632934 |
Epoch 89: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.15260987382035 |
Epoch 90: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.15786939053922 |
Epoch 91: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.16260463267833 |
Epoch 92: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.166868983088314 |
Epoch 93: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.170706041582356 |
Epoch 94: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.17415773546374 |
Epoch 95: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.17726405604824 |
Epoch 96: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.18006695355023 |
Epoch 97: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.18258318743548 |
Epoch 98: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.184848510467255 |
Epoch 99: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Valid
ation loss: 63.18688586810688 |
Epoch 100: Train err: 0.0, Train loss: 0.0 |Validation err: 0.8798798798798799, Vali
dation loss: 63.18872056565843 |
Finished Training
Total time elapsed: 290.33 seconds
Accuracy on batch of 64:  1.0
```

From the above results, The accuracy is 1.0, showing clear signs of overfitting due to the model "memorizing" the small dataset. Below is the training curve:

In [30]:
```
sanity_path = get_model_name(sanity_model.name, batch_size=64, learning_rate=0.001,
plot_training_curve(sanity_path)
```


Train vs Validation Error
Train
Validation
Error
Epoch
1.0
0.8
0.6
0.4
0.2
0.0
0
20
40
60
80
100


Train vs Validation Loss
Train
Validation
Loss
Epoch
60
50
40
30
20
10
0
0
20
40
60
80
100

3. Hyperparameter Search [15 pt]

Part (a) - 3 pt

List 3 hyperparameters that you think are most worth tuning. Choose at least one hyperparameter related to the model architecture.

Hyperparameters to tune:

1. batch_size
2. learning_rate
3. the number of layers in CNN structure

Part (b) - 5 pt

Tune the hyperparameters you listed in Part (a), trying as many values as you need to until you feel satisfied that you are getting a good model. Plot the training curve of at least 4 different hyperparameter settings.

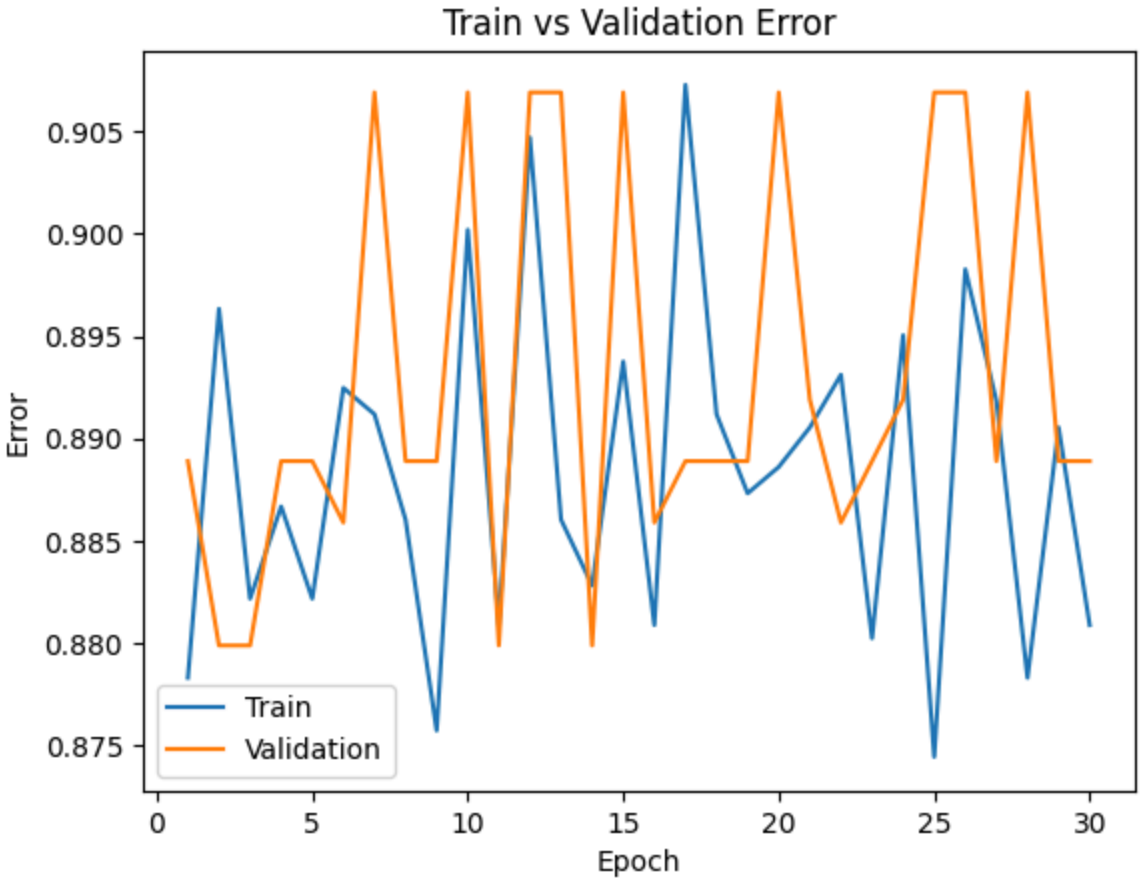
1st setting: Default settings

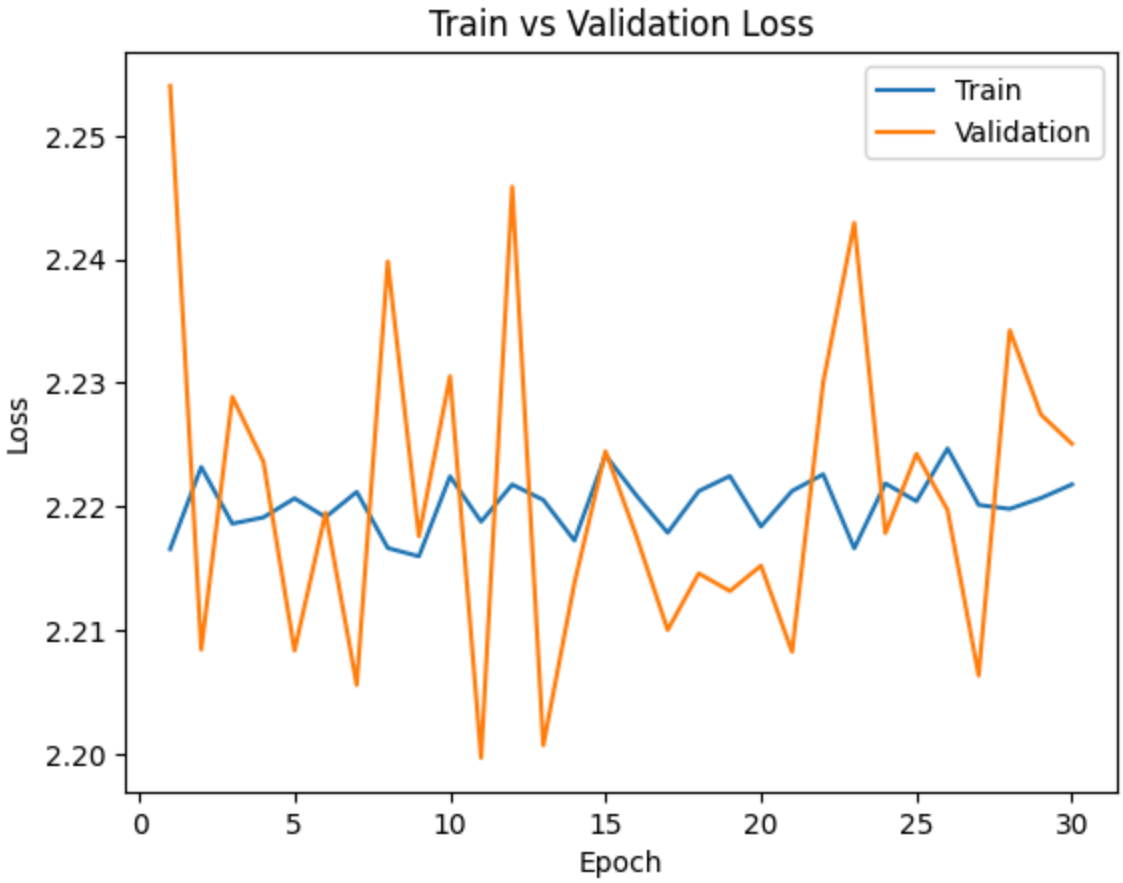
1. batch_size = 64
2. learning_rate = 0.01
3. number of convolution layers in CNN = 2

In [27]:
```
cnn = CNN()
train_net(cnn, train_loader=train_loader, val_loader=val_loader, batch_size=64, lea
cnn_path = get_model_name(cnn.name, batch_size=64, learning_rate=0.01, epoch=29)
plot_training_curve(cnn_path)
```

```
Epoch 1: Train err: 0.8783000643915003, Train loss: 2.2165692088224778 |Validation e
rr: 0.8888888888888888, Validation loss: 2.254027973782193 |Validation acc: 0.111111
11111111116
Epoch 2: Train err: 0.8963296844816484, Train loss: 2.2232047981089034 |Validation e
rr: 0.8798798798798799, Validation loss: 2.2084393107497298 |Validation acc: 0.12012
012012012008
Epoch 3: Train err: 0.8821635544108177, Train loss: 2.218625932298625 |Validation er
r: 0.8798798798798799, Validation loss: 2.2288800903984733 |Validation acc: 0.120120
12012012008
Epoch 4: Train err: 0.8866709594333548, Train loss: 2.2191334357970924 |Validation e
rr: 0.8888888888888888, Validation loss: 2.2236431776224315 |Validation acc: 0.11111
111111111116
Epoch 5: Train err: 0.8821635544108177, Train loss: 2.2206723348448065 |Validation e
rr: 0.8888888888888888, Validation loss: 2.208358196524886 |Validation acc: 0.111111
11111111116
Epoch 6: Train err: 0.892466194462331, Train loss: 2.219152735573817 |Validation er
r: 0.8858858858858859, Validation loss: 2.2195126108221106 |Validation acc: 0.114114
1141141141
Epoch 7: Train err: 0.8911783644558918, Train loss: 2.221196620063634 |Validation er
r: 0.9069069069069069, Validation loss: 2.2055811828321166 |Validation acc: 0.093093
0930930931
Epoch 8: Train err: 0.8860270444301352, Train loss: 2.216654696697891 |Validation er
r: 0.8888888888888888, Validation loss: 2.2398233814640447 |Validation acc: 0.111111
11111111116
Epoch 9: Train err: 0.875724404378622, Train loss: 2.215983689407034 |Validation er
r: 0.8888888888888888, Validation loss: 2.2176220789327994 |Validation acc: 0.111111
11111111116
Epoch 10: Train err: 0.9001931745009659, Train loss: 2.2224506039198797 |Validation
err: 0.9069069069069069, Validation loss: 2.230550278056491 |Validation acc: 0.09309
30930930931
Epoch 11: Train err: 0.8808757244043787, Train loss: 2.218784946130309 |Validation e
rr: 0.8798798798798799, Validation loss: 2.1996896123742915 |Validation acc: 0.12012
012012012008
Epoch 12: Train err: 0.9047005795235029, Train loss: 2.2217900616387283 |Validation
err: 0.9069069069069069, Validation loss: 2.2458692939431817 |Validation acc: 0.0930
930930930931
Epoch 13: Train err: 0.8860270444301352, Train loss: 2.220546433408262 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2007029221222565 |Validation acc: 0.09309
30930930931
Epoch 14: Train err: 0.8828074694140373, Train loss: 2.2172625447117893 |Validation
err: 0.8798798798798799, Validation loss: 2.2138110581819004 |Validation acc: 0.1201
2012012012008
Epoch 15: Train err: 0.8937540244687702, Train loss: 2.2241621061977233 |Validation
err: 0.9069069069069069, Validation loss: 2.224474333070062 |Validation acc: 0.09309
30930930931
Epoch 16: Train err: 0.8808757244043787, Train loss: 2.220890193160549 |Validation e
rr: 0.8858858858858859, Validation loss: 2.217574340803129 |Validation acc: 0.114114
1141141141
Epoch 17: Train err: 0.9072762395363811, Train loss: 2.2178926323585486 |Validation
err: 0.8888888888888888, Validation loss: 2.21002828179895 |Validation acc: 0.111111
11111111116
Epoch 18: Train err: 0.8911783644558918, Train loss: 2.221261652146933 |Validation e
rr: 0.8888888888888888, Validation loss: 2.214602951530938 |Validation acc: 0.111111
11111111116
Epoch 19: Train err: 0.8873148744365744, Train loss: 2.2224747700915364 |Validation
err: 0.8888888888888888, Validation loss: 2.2131731997023114 |Validation acc: 0.1111
```

```
1111111111116
Epoch 20: Train err: 0.8886027044430135, Train loss: 2.2183959266100097 |Validation
err: 0.9069069069069069, Validation loss: 2.2152299981217483 |Validation acc: 0.0930
930930930931
Epoch 21: Train err: 0.8905344494526722, Train loss: 2.2212675110570705 |Validation
err: 0.8918918918918919, Validation loss: 2.2082629805212624 |Validation acc: 0.1081
0810810810811
Epoch 22: Train err: 0.8931101094655506, Train loss: 2.2226191853525097 |Validation
err: 0.8858858858858859, Validation loss: 2.230035660861133 |Validation acc: 0.11411
41141141141
Epoch 23: Train err: 0.8802318094011591, Train loss: 2.216632502891138 |Validation e
rr: 0.8888888888888888, Validation loss: 2.2429457877849317 |Validation acc: 0.11111
111111111116
Epoch 24: Train err: 0.8950418544752092, Train loss: 2.221878537115403 |Validation e
rr: 0.8918918918918919, Validation loss: 2.2178589946872838 |Validation acc: 0.10810
810810810811
Epoch 25: Train err: 0.8744365743721829, Train loss: 2.2204371181059406 |Validation
err: 0.9069069069069069, Validation loss: 2.2242712566444465 |Validation acc: 0.0930
930930930931
Epoch 26: Train err: 0.8982614294913072, Train loss: 2.2247030609128404 |Validation
err: 0.9069069069069069, Validation loss: 2.219717824781263 |Validation acc: 0.09309
30930930931
Epoch 27: Train err: 0.8918222794591114, Train loss: 2.2201395134578883 |Validation
err: 0.8888888888888888, Validation loss: 2.206346135955673 |Validation acc: 0.11111
111111111116
Epoch 28: Train err: 0.8783000643915003, Train loss: 2.21981808629561 |Validation er
r: 0.9069069069069069, Validation loss: 2.234270532925924 |Validation acc: 0.0930930
930930931
Epoch 29: Train err: 0.8905344494526722, Train loss: 2.2206938515458656 |Validation
err: 0.8888888888888888, Validation loss: 2.227451638774471 |Validation acc: 0.11111
111111111116
Epoch 30: Train err: 0.8808757244043787, Train loss: 2.2217952643988292 |Validation
err: 0.8888888888888888, Validation loss: 2.225086482795509 |Validation acc: 0.11111
111111111116
Finished Training
Total time elapsed: 541.10 seconds
```





2nd setting: Change batch_size

1. batch_size = 256

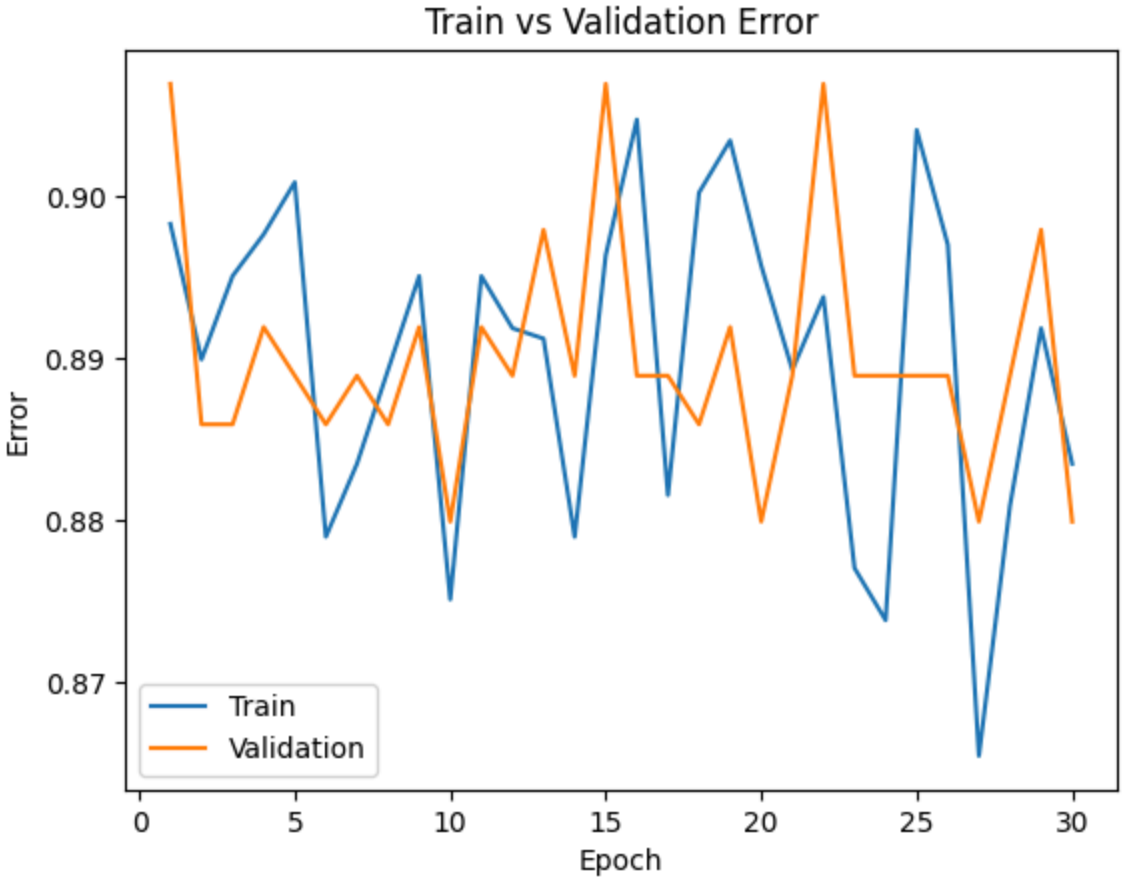
2. learning_rate = 0.01

3. number of convolution layers in CNN = 2

In [31]:
```
cnn = CNN()
train_net(cnn, train_loader=train_loader, val_loader=val_loader, batch_size=256, le
cnn_path = get_model_name(cnn.name, batch_size=256, learning_rate=0.01, epoch=29)
plot_training_curve(cnn_path)
```

```
Epoch 1: Train err: 0.8982614294913072, Train loss: 2.2171738244454016 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2273346275180668 |
Epoch 2: Train err: 0.8898905344494527, Train loss: 2.2219655654235417 |Validation e
rr: 0.8858858858858859, Validation loss: 2.2375867502825395 |
Epoch 3: Train err: 0.8950418544752092, Train loss: 2.220596656519908 |Validation er
r: 0.8858858858858859, Validation loss: 2.2167141809835806 |
Epoch 4: Train err: 0.8976175144880876, Train loss: 2.2210300070043543 |Validation e
rr: 0.8918918918918919, Validation loss: 2.2217249612550476 |
Epoch 5: Train err: 0.9008370895041854, Train loss: 2.2223926484009104 |Validation e
rr: 0.8888888888888888, Validation loss: 2.207928492500259 |
Epoch 6: Train err: 0.8789439793947199, Train loss: 2.221174585765512 |Validation er
r: 0.8858858858858859, Validation loss: 2.2278687141321085 |
Epoch 7: Train err: 0.8834513844172569, Train loss: 2.2138496034773256 |Validation e
rr: 0.8888888888888888, Validation loss: 2.222909361392528 |
Epoch 8: Train err: 0.8892466194462331, Train loss: 2.2213545163370148 |Validation e
rr: 0.8858858858858859, Validation loss: 2.219541813876178 |
Epoch 9: Train err: 0.8950418544752092, Train loss: 2.2165025690641236 |Validation e
rr: 0.8918918918918919, Validation loss: 2.226020287465047 |
Epoch 10: Train err: 0.8750804893754024, Train loss: 2.218392164109986 |Validation e
rr: 0.8798798798798799, Validation loss: 2.234232118537834 |
Epoch 11: Train err: 0.8950418544752092, Train loss: 2.2242289842978953 |Validation
err: 0.8918918918918919, Validation loss: 2.219771766805792 |
Epoch 12: Train err: 0.8918222794591114, Train loss: 2.2230236144197884 |Validation
err: 0.8888888888888888, Validation loss: 2.220349119948195 |
Epoch 13: Train err: 0.8911783644558918, Train loss: 2.2183451479815393 |Validation
err: 0.8978978978978979, Validation loss: 2.2167269854216247 |
Epoch 14: Train err: 0.8789439793947199, Train loss: 2.2208596482556326 |Validation
err: 0.8888888888888888, Validation loss: 2.222542092248842 |
Epoch 15: Train err: 0.8963296844816484, Train loss: 2.2239971024254714 |Validation
err: 0.9069069069069069, Validation loss: 2.2237642853109687 |
Epoch 16: Train err: 0.9047005795235029, Train loss: 2.219430772396955 |Validation e
rr: 0.8888888888888888, Validation loss: 2.243961222536929 |
Epoch 17: Train err: 0.8815196394075981, Train loss: 2.221006718053406 |Validation e
rr: 0.8888888888888888, Validation loss: 2.235545954188785 |
Epoch 18: Train err: 0.9001931745009659, Train loss: 2.2227578021907375 |Validation
err: 0.8858858858858859, Validation loss: 2.214856311007663 |
Epoch 19: Train err: 0.9034127495170637, Train loss: 2.2177321375836425 |Validation
err: 0.8918918918918919, Validation loss: 2.2264865840877497 |
Epoch 20: Train err: 0.8956857694784288, Train loss: 2.2179870720609727 |Validation
err: 0.8798798798798799, Validation loss: 2.2149441242218018 |
Epoch 21: Train err: 0.8892466194462331, Train loss: 2.2174623651191334 |Validation
err: 0.8888888888888888, Validation loss: 2.20944989121354 |
Epoch 22: Train err: 0.8937540244687702, Train loss: 2.224542784598744 |Validation e
rr: 0.9069069069069069, Validation loss: 2.22459810130947 |
Epoch 23: Train err: 0.8770122343850612, Train loss: 2.2154717831479607 |Validation
err: 0.8888888888888888, Validation loss: 2.221453540318005 |
Epoch 24: Train err: 0.8737926593689633, Train loss: 2.217981730440856 |Validation e
rr: 0.8888888888888888, Validation loss: 2.2321010584587806 |
Epoch 25: Train err: 0.9040566645202833, Train loss: 2.2246567171309275 |Validation
err: 0.8888888888888888, Validation loss: 2.213801674298696 |
Epoch 26: Train err: 0.896973599484868, Train loss: 2.220295928481004 |Validation er
r: 0.8888888888888888, Validation loss: 2.2074951681646855 |
Epoch 27: Train err: 0.8654217643271088, Train loss: 2.2140759584294236 |Validation
err: 0.8798798798798799, Validation loss: 2.2240815699637473 |
Epoch 28: Train err: 0.8808757244043787, Train loss: 2.222915383822213 |Validation e
rr: 0.8888888888888888, Validation loss: 2.225458903713627 |
```

```
Epoch 29: Train err: 0.8918222794591114, Train loss: 2.221352781316041 |Validation e
rr: 0.8978978978978979, Validation loss: 2.2239778643255836 |
Epoch 30: Train err: 0.8834513844172569, Train loss: 2.2171495914152186 |Validation
err: 0.8798798798798799, Validation loss: 2.2148216608408333 |
Finished Training
Total time elapsed: 489.40 seconds
```



3rd setting: Change learning_rate

1. batch_size = 64
2. learning_rate = 0.001
3. number of convolution layers in CNN = 2

In [32]:
```
cnn = CNN()
train_net(cnn, train_loader=train_loader, val_loader=val_loader, batch_size=64, lea
cnn_path = get_model_name(cnn.name, batch_size=64, learning_rate=0.001, epoch=29)
plot_training_curve(cnn_path)
```

```
Epoch 1: Train err: 0.8853831294269157, Train loss: 2.2020989990357194 |Validation e
rr: 0.9069069069069069, Validation loss: 2.204698958554425 |
Epoch 2: Train err: 0.8770122343850612, Train loss: 2.1996282588568183 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2025719963394486 |
Epoch 3: Train err: 0.8821635544108177, Train loss: 2.1993791449093005 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2016308407883742 |
Epoch 4: Train err: 0.8931101094655506, Train loss: 2.1992254444498136 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2024956250691914 |
Epoch 5: Train err: 0.8834513844172569, Train loss: 2.1994502251177392 |Validation e
rr: 0.9069069069069069, Validation loss: 2.203377821066 |
Epoch 6: Train err: 0.8821635544108177, Train loss: 2.199536185838757 |Validation er
r: 0.9069069069069069, Validation loss: 2.202592723004453 |
Epoch 7: Train err: 0.892466194462331, Train loss: 2.1993151920807414 |Validation er
r: 0.9069069069069069, Validation loss: 2.204333500819163 |
Epoch 8: Train err: 0.8866709594333548, Train loss: 2.1997540864496177 |Validation e
rr: 0.9069069069069069, Validation loss: 2.205555592213307 |
Epoch 9: Train err: 0.8866709594333548, Train loss: 2.1993078640485226 |Validation e
rr: 0.9069069069069069, Validation loss: 2.204022538554561 |
Epoch 10: Train err: 0.8834513844172569, Train loss: 2.1989719529191833 |Validation
err: 0.8978978978978979, Validation loss: 2.2029905576963684 |
Epoch 11: Train err: 0.892466194462331, Train loss: 2.1993003867935994 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2050234426607243 |
Epoch 12: Train err: 0.8892466194462331, Train loss: 2.199463291622636 |Validation e
rr: 0.9069069069069069, Validation loss: 2.203874847194454 |
Epoch 13: Train err: 0.8815196394075981, Train loss: 2.199486866046441 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2030705088251703 |
Epoch 14: Train err: 0.8931101094655506, Train loss: 2.199557193386424 |Validation e
rr: 0.9069069069069069, Validation loss: 2.203295258788375 |
Epoch 15: Train err: 0.8815196394075981, Train loss: 2.198674435855033 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2009086415574357 |
Epoch 16: Train err: 0.892466194462331, Train loss: 2.1990885472036683 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2006634069276645 |
Epoch 17: Train err: 0.8853831294269157, Train loss: 2.1992193105062667 |Validation
err: 0.9069069069069069, Validation loss: 2.2035598153466576 |
Epoch 18: Train err: 0.8840952994204765, Train loss: 2.1991739587789954 |Validation
err: 0.9069069069069069, Validation loss: 2.2009519363666796 |
Epoch 19: Train err: 0.8853831294269157, Train loss: 2.1993037738572836 |Validation
err: 0.9069069069069069, Validation loss: 2.202004986124354 |
Epoch 20: Train err: 0.8886027044430135, Train loss: 2.1995310872428586 |Validation
err: 0.9069069069069069, Validation loss: 2.2032325940805153 |
Epoch 21: Train err: 0.8789439793947199, Train loss: 2.1978293183689646 |Validation
err: 0.8918918918918919, Validation loss: 2.2063578148861906 |
Epoch 22: Train err: 0.9001931745009659, Train loss: 2.1995893650030367 |Validation
err: 0.8918918918918919, Validation loss: 2.2040276412849313 |
Epoch 23: Train err: 0.8847392144236961, Train loss: 2.199621362526495 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2007147721700124 |
Epoch 24: Train err: 0.8821635544108177, Train loss: 2.1989306204563097 |Validation
err: 0.9069069069069069, Validation loss: 2.203144226704274 |
Epoch 25: Train err: 0.8866709594333548, Train loss: 2.199670106920978 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2042318493038326 |
Epoch 26: Train err: 0.8853831294269157, Train loss: 2.1990325526122656 |Validation
err: 0.9069069069069069, Validation loss: 2.2057207776261523 |
Epoch 27: Train err: 0.8860270444301352, Train loss: 2.199753579906088 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2045061752960846 |
Epoch 28: Train err: 0.8802318094011591, Train loss: 2.1994636889357606 |Validation
err: 0.9069069069069069, Validation loss: 2.2047142409705542 |
```

```
Epoch 29: Train err: 0.8860270444301352, Train loss: 2.1991876924874623 |Validation
err: 0.9069069069069069, Validation loss: 2.2011855908700295 |
Epoch 30: Train err: 0.8886027044430135, Train loss: 2.199551136971134 |Validation e
rr: 0.9069069069069069, Validation loss: 2.204785975608024 |
Finished Training
Total time elapsed: 569.27 seconds
```

4th setting: Add another convolution layer

1. batch_size = 64
2. learning_rate = 0.01
3. number of convolution layers in CNN = 3

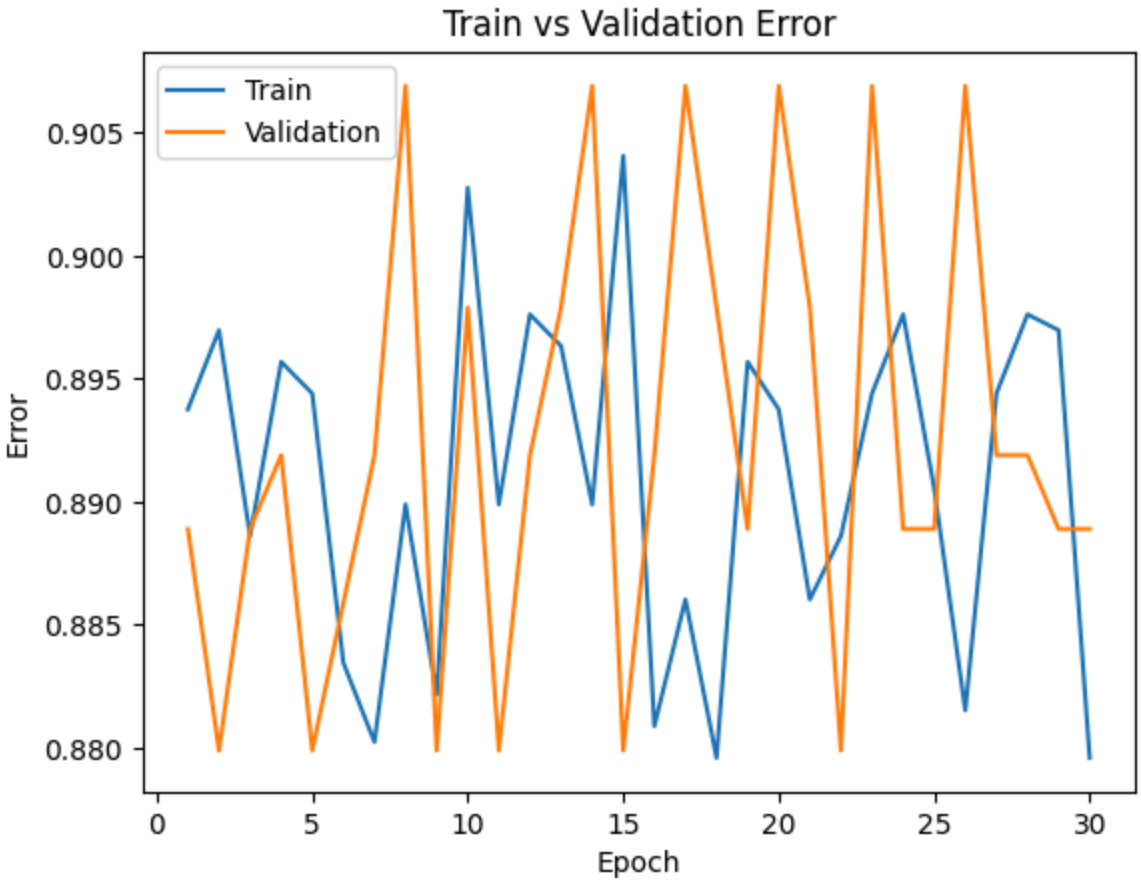
In [27]:
```
class CNN3(nn.Module):
    def __init__(self):
        super(CNN3, self).__init__()
        self.name = "cnn3"

        # Convolution layers
        self.conv1 = nn.Conv2d(3, 5, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(5, 10, 5)
        self.conv3 = nn.Conv2d(10, 20, 5) # New 3rd layer

        # Fully connected layers
        self.fc1 = nn.Linear(20 * 24 * 24, 32)
        self.fc2 = nn.Linear(32, 9)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x))) # Pass through new layer
```

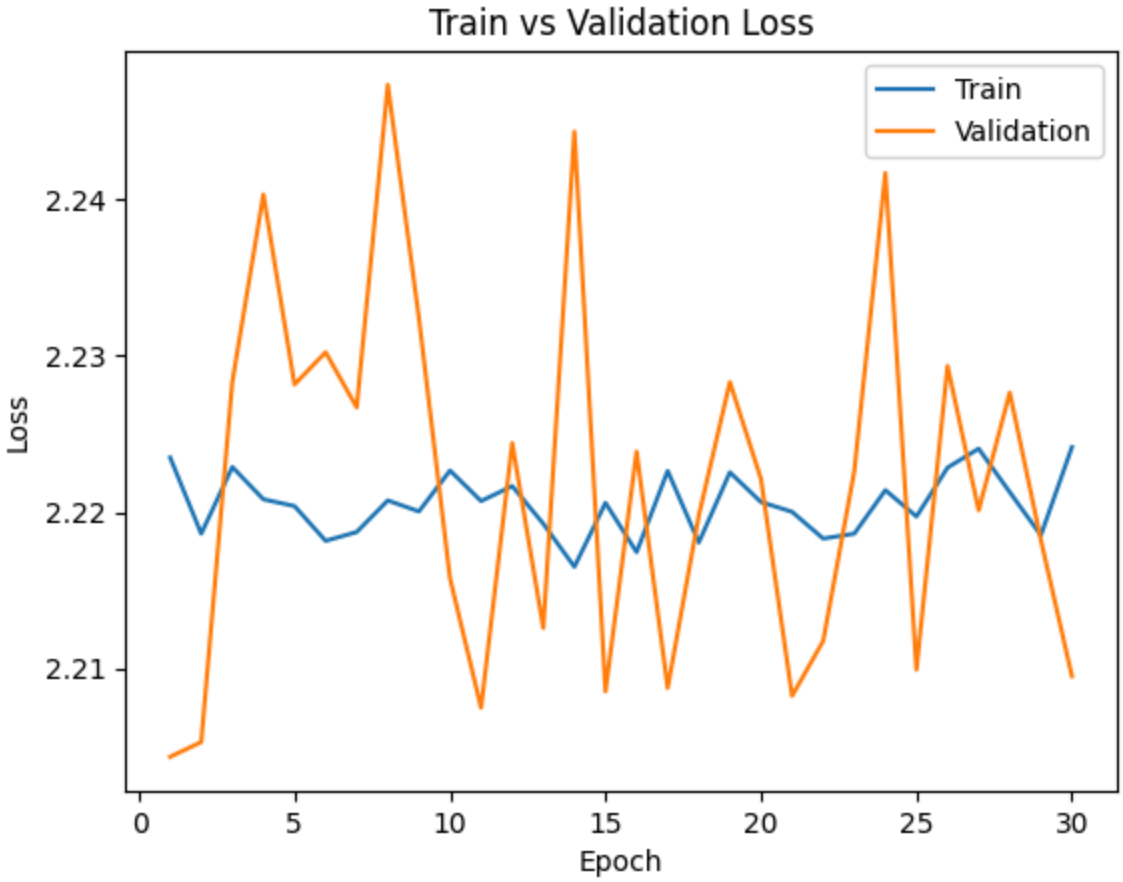
```
        x = x.view(-1, 20 * 24 * 24)

        x = F.relu(self.fc1(x))
        x = self.fc2(x)

        return x
```

In [ ]:
```
cnn3 = CNN3()
train_net(cnn3, train_loader=train_loader, val_loader=val_loader, batch_size=64, le
cnn3_path = get_model_name(cnn3.name, batch_size=64, learning_rate=0.01, epoch=29)
plot_training_curve(cnn3_path)
```

```
Epoch 1: Train err: 0.8937540244687702, Train loss: 2.223492400230167 |Validation er
r: 0.8888888888888888, Validation loss: 2.204331952172357 |
Epoch 2: Train err: 0.896973599484868, Train loss: 2.2186178180838274 |Validation er
r: 0.8798798798798799, Validation loss: 2.2052795307056323 |
Epoch 3: Train err: 0.8886027044430135, Train loss: 2.222882499252683 |Validation er
r: 0.8888888888888888, Validation loss: 2.2283353071670993 |
Epoch 4: Train err: 0.8956857694784288, Train loss: 2.2208161050092765 |Validation e
rr: 0.8918918918918919, Validation loss: 2.240330943832168 |
Epoch 5: Train err: 0.8943979394719896, Train loss: 2.220382590257039 |Validation er
r: 0.8798798798798799, Validation loss: 2.228167120758836 |
Epoch 6: Train err: 0.8834513844172569, Train loss: 2.2181589287170196 |Validation e
rr: 0.8858858858858859, Validation loss: 2.2302302941903696 |
Epoch 7: Train err: 0.8802318094011591, Train loss: 2.2187225450182915 |Validation e
rr: 0.8918918918918919, Validation loss: 2.2266963938693025 |
Epoch 8: Train err: 0.8898905344494527, Train loss: 2.2207432658305417 |Validation e
rr: 0.9069069069069069, Validation loss: 2.247347638055727 |
Epoch 9: Train err: 0.8821635544108177, Train loss: 2.2200405999144968 |Validation e
rr: 0.8798798798798799, Validation loss: 2.2324259109325237 |
Epoch 10: Train err: 0.9027688345138442, Train loss: 2.222659840632928 |Validation e
rr: 0.8978978978978979, Validation loss: 2.215760022670299 |
Epoch 11: Train err: 0.8898905344494527, Train loss: 2.2207057157487005 |Validation
err: 0.8798798798798799, Validation loss: 2.207469998895227 |
Epoch 12: Train err: 0.8976175144880876, Train loss: 2.22166420817759 |Validation er
r: 0.8918918918918919, Validation loss: 2.2244188037362544 |
Epoch 13: Train err: 0.8963296844816484, Train loss: 2.219251480857864 |Validation e
rr: 0.8978978978978979, Validation loss: 2.2125807216575555 |
Epoch 14: Train err: 0.8898905344494527, Train loss: 2.216497710652299 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2443530233772666 |
Epoch 15: Train err: 0.9040566645202833, Train loss: 2.220588605632647 |Validation e
rr: 0.8798798798798799, Validation loss: 2.208520393829804 |
Epoch 16: Train err: 0.8808757244043787, Train loss: 2.2174349266409643 |Validation
err: 0.8918918918918919, Validation loss: 2.223868649285119 |
Epoch 17: Train err: 0.8860270444301352, Train loss: 2.2226327492664186 |Validation
err: 0.9069069069069069, Validation loss: 2.2087447582422435 |
Epoch 18: Train err: 0.8795878943979395, Train loss: 2.2180491662839112 |Validation
err: 0.8978978978978979, Validation loss: 2.2197739383479855 |
Epoch 19: Train err: 0.8956857694784288, Train loss: 2.222542952124871 |Validation e
rr: 0.8888888888888888, Validation loss: 2.2283209505023898 |
Epoch 20: Train err: 0.8937540244687702, Train loss: 2.220644068733309 |Validation e
rr: 0.9069069069069069, Validation loss: 2.222122151930411 |
Epoch 21: Train err: 0.8860270444301352, Train loss: 2.220015374906433 |Validation e
rr: 0.8978978978978979, Validation loss: 2.208233236192583 |
Epoch 22: Train err: 0.8886027044430135, Train loss: 2.218304753303528 |Validation e
rr: 0.8798798798798799, Validation loss: 2.2117494777874187 |
Epoch 23: Train err: 0.8943979394719896, Train loss: 2.2186154154754805 |Validation
err: 0.9069069069069069, Validation loss: 2.222604826047972 |
Epoch 24: Train err: 0.8976175144880876, Train loss: 2.221402407383351 |Validation e
rr: 0.8888888888888888, Validation loss: 2.2417057229949906 |
Epoch 25: Train err: 0.8905344494526722, Train loss: 2.219715509964279 |Validation e
rr: 0.8888888888888888, Validation loss: 2.2099060281976923 |
Epoch 26: Train err: 0.8815196394075981, Train loss: 2.222847512208947 |Validation e
rr: 0.9069069069069069, Validation loss: 2.2293479918717622 |
Epoch 27: Train err: 0.8943979394719896, Train loss: 2.2240579797618247 |Validation
err: 0.8918918918918919, Validation loss: 2.2201157070852973 |
Epoch 28: Train err: 0.8976175144880876, Train loss: 2.221274926135867 |Validation e
rr: 0.8918918918918919, Validation loss: 2.2276510265138416 |
```

```
Epoch 29: Train err: 0.896973599484868, Train loss: 2.2185158119306054 |Validation e
rr: 0.8888888888888888, Validation loss: 2.218218716773185 |
Epoch 30: Train err: 0.8795878943979395, Train loss: 2.22416118829387 |Validation er
r: 0.8888888888888888, Validation loss: 2.209492746058169 |
Finished Training
Total time elapsed: 499.41 seconds
```





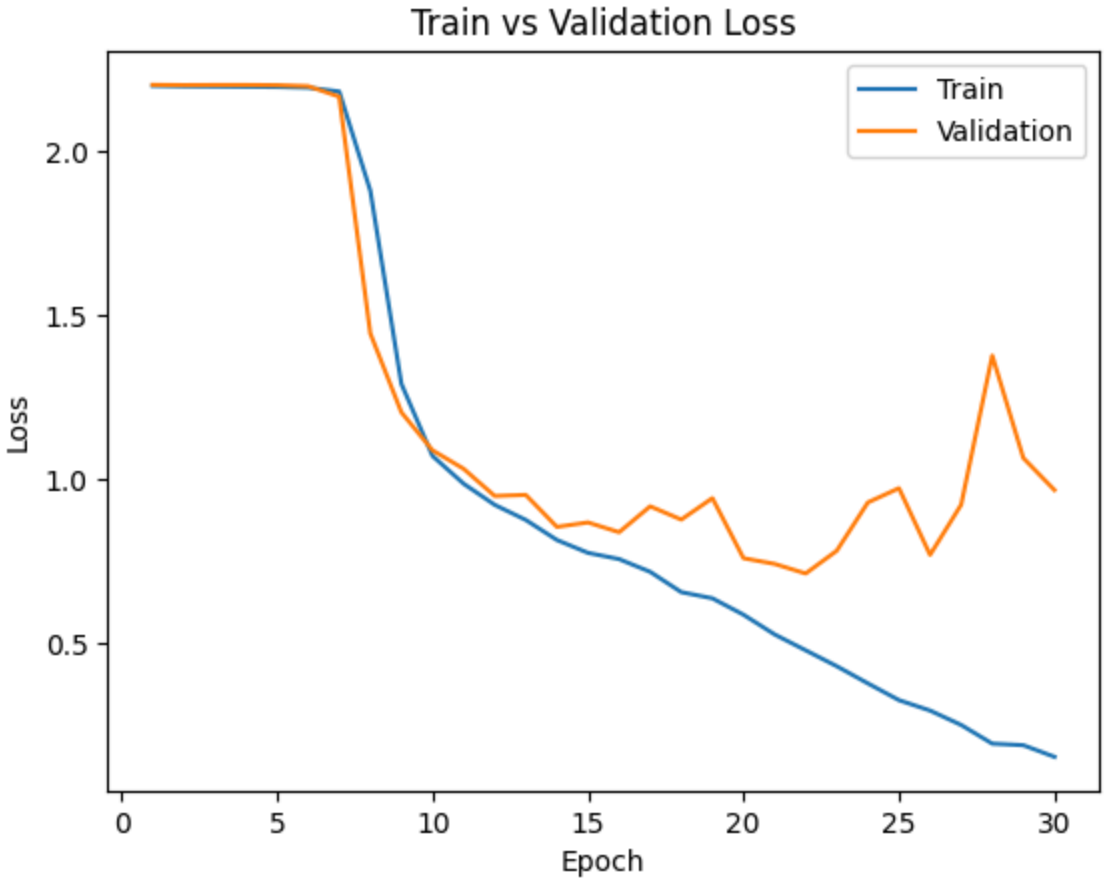
5th setting: Improve all paramaters

1. batch_size = 64
2. learning_rate = 0.0001
3. number of convolution layers in CNN = 3

In [ ]:
```
cnn3 = CNN3()
train_net(cnn3, train_loader=train_loader, val_loader=val_loader, batch_size=64, le
cnn3_path = get_model_name(cnn3.name, batch_size=64, learning_rate=0.0001, epoch=29
plot_training_curve(cnn3_path)
```

```
Epoch 1: Train err: 0.8873148744365744, Train loss: 2.199451100419432 |Validation er
r: 0.8888888888888888, Validation loss: 2.2024430376631363 |
Epoch 2: Train err: 0.8892466194462331, Train loss: 2.197842040525585 |Validation er
r: 0.8888888888888888, Validation loss: 2.201640673227854 |
Epoch 3: Train err: 0.8879587894397939, Train loss: 2.1972174197723233 |Validation e
rr: 0.8888888888888888, Validation loss: 2.2019775551002665 |
Epoch 4: Train err: 0.8847392144236961, Train loss: 2.196561997426531 |Validation er
r: 0.9069069069069069, Validation loss: 2.201979910647189 |
Epoch 5: Train err: 0.8808757244043787, Train loss: 2.1959333043672618 |Validation e
rr: 0.9069069069069069, Validation loss: 2.201419084279745 |
Epoch 6: Train err: 0.8731487443657437, Train loss: 2.1937619881405803 |Validation e
rr: 0.9069069069069069, Validation loss: 2.1983778627069146 |
Epoch 7: Train err: 0.8177720540888602, Train loss: 2.1821849165005753 |Validation e
rr: 0.8288288288288288, Validation loss: 2.166388767617601 |
Epoch 8: Train err: 0.6870573084352866, Train loss: 1.879829514351915 |Validation er
r: 0.5075075075075075, Validation loss: 1.4447810037268534 |
Epoch 9: Train err: 0.45717965228589824, Train loss: 1.2908554098674403 |Validation
err: 0.37537537537537535, Validation loss: 1.2041856163872164 |
Epoch 10: Train err: 0.3676754668383773, Train loss: 1.0713271346803472 |Validation
err: 0.37237237237237236, Validation loss: 1.088475983220671 |
Epoch 11: Train err: 0.3425627817128139, Train loss: 0.9867315012590425 |Validation
err: 0.35435435435435436, Validation loss: 1.0321877690448318 |
Epoch 12: Train err: 0.313586606567933, Train loss: 0.9222582283046762 |Validation e
rr: 0.33933933933933935, Validation loss: 0.9493510958403106 |
Epoch 13: Train err: 0.313586606567933, Train loss: 0.8762596500917234 |Validation e
rr: 0.3153153153153153, Validation loss: 0.9526036241529088 |
Epoch 14: Train err: 0.2736638763683194, Train loss: 0.8151440606719168 |Validation
err: 0.2672672672672673, Validation loss: 0.8551434499760846 |
Epoch 15: Train err: 0.26142949130714743, Train loss: 0.7759251119956501 |Validation
err: 0.2822822822822823, Validation loss: 0.868607050918985 |
Epoch 16: Train err: 0.25820991629104956, Train loss: 0.7569138318444948 |Validation
err: 0.26126126126126126, Validation loss: 0.8387923082183589 |
Epoch 17: Train err: 0.2421120412105602, Train loss: 0.7180819378279026 |Validation
err: 0.25825825825825827, Validation loss: 0.9177580972933389 |
Epoch 18: Train err: 0.23116548615582744, Train loss: 0.6561900828208238 |Validation
err: 0.2672672672672673, Validation loss: 0.8773621465285145 |
Epoch 19: Train err: 0.22794591113972956, Train loss: 0.637428701653113 |Validation
err: 0.3153153153153153, Validation loss: 0.9426732534462177 |
Epoch 20: Train err: 0.20090148100450742, Train loss: 0.5874446446952774 |Validation
err: 0.25225225225225223, Validation loss: 0.7596393545375961 |
Epoch 21: Train err: 0.1854475209272376, Train loss: 0.5273871710391768 |Validation
err: 0.22822822822822822, Validation loss: 0.7422950971558189 |
Epoch 22: Train err: 0.1725692208628461, Train loss: 0.4789234898703952 |Validation
err: 0.19519519519519518, Validation loss: 0.7129836715218623 |
Epoch 23: Train err: 0.15904700579523504, Train loss: 0.43037938837951145 |Validatio
n err: 0.2072072072072072, Validation loss: 0.7824439802533677 |
Epoch 24: Train err: 0.13007083065035416, Train loss: 0.37813038938763494 |Validatio
n err: 0.24024024024024024, Validation loss: 0.9295751215277044 |
Epoch 25: Train err: 0.1107533805537669, Train loss: 0.3269925704019777 |Validation
err: 0.2552552552552553, Validation loss: 0.973003919478588 |
Epoch 26: Train err: 0.10431423052157116, Train loss: 0.295264869049397 |Validation
err: 0.1981981981981982, Validation loss: 0.7698433660187038 |
Epoch 27: Train err: 0.07726980038634901, Train loss: 0.2513452256410737 |Validation
err: 0.2042042042042042, Validation loss: 0.9226792357996955 |
Epoch 28: Train err: 0.0592401802962009, Train loss: 0.19447559047159588 |Validation
err: 0.2672672672672673, Validation loss: 1.3772582634947856 |
```

```
Epoch 29: Train err: 0.06310367031551835, Train loss: 0.18967893711561765 |Validatio
n err: 0.21021021021021022, Validation loss: 1.0648412885622835 |
Epoch 30: Train err: 0.054732775273663874, Train loss: 0.15422849350053375 |Validati
on err: 0.18618618618618618, Validation loss: 0.9674896201681038 |
Finished Training
Total time elapsed: 614.48 seconds
```



Part (c) - 3 pt

Choose the best model out of all the ones that you have trained. Justify your choice.

The best model has the following parameters:

1. batch_size = 64
2. learning_rate = 0.0001
3. number of convolution layers in CNN = 3

For the batch size, after experimenting between 64 and 256, the results showed that there isn't much improvement between the two options. However, a batch size that is too big can require a lot of memory to process high-resolution images and cause the optimizer to take broad steps. A batch size of 64 introduces just enough random noise, and takes smaller steps to avoid accidentally missing the "sweet spot".

For the learning rate, just simply changing from 0.01 to 0.001 showed that decreasing the learning rate improved the model's performance. Lowering the learning rate when using smaller batches can help ensure the model doesn't over-correct itself. Choosing 0.0001 will force the model to learn slower which is shown by a smoother loss curve.

For the architecture of the model, processing high resolution images typically work better with 3 or 4 layers. With this combination of parameters, I am happy with the training and validation results so far.

Part (d) - 4 pt

Report the test accuracy of your best model. You should only do this step once and prior to this step you should have only used the training and validation data.

In [16]:
```
best_model = CNN3()
best_model_path = "Model_data/model_{0}_bs{1}_lr{2}_epoch{3}".format("cnn3", 64, 0.
state = torch.load(best_model_path)
best_model.load_state_dict(state)

best_model.eval() # Ensure model is in testing mode and not training mode

correct = 0
total = 0
with torch.no_grad():
    for imgs, labels in test_loader:
        output = best_model(imgs)
        pred = output.max(1, keepdim=True)[1]
        correct += pred.eq(labels.view_as(pred)).sum().item()
        total += imgs.shape[0]

print('Final Test Accuracy: ', correct / total)
```

```
Final Test Accuracy:  0.7957957957957958
```

4. Transfer Learning [15 pt]

For many image classification tasks, it is generally not a good idea to train a very large deep neural network model from scratch due to the enormous compute requirements and lack of sufficient amounts of training data.

One of the better options is to try using an existing model that performs a similar task to the one you need to solve. This method of utilizing a pre-trained network for other similar tasks is broadly termed **Transfer Learning**. In this assignment, we will use Transfer Learning to extract features from the hand gesture images. Then, train a smaller network to use these features as input and classify the hand gestures.

As you have learned from the CNN lecture, convolution layers extract various features from the images which get utilized by the fully connected layers for correct classification. AlexNet architecture played a pivotal role in establishing Deep Neural Nets as a go-to tool for image classification problems and we will use an ImageNet pre-trained AlexNet model to extract features in this assignment.

Part (a) - 5 pt

Here is the code to load the AlexNet network, with pretrained weights. When you first run the code, PyTorch will download the pretrained weights from the internet.

In [17]:
```
import torchvision.models
alexnet = torchvision.models.alexnet(pretrained=True)
```

```
c:\Users\sissi\OneDrive\Desktop\Code\APS360\APS360\venv\Lib\site-packages\torchvision\models\_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
c:\Users\sissi\OneDrive\Desktop\Code\APS360\APS360\venv\Lib\site-packages\torchvision\models\_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=AlexNet_Weights.IMAGENET1K_V1`. You can also use `weights=AlexNet_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)
0.1%
Downloading: "https://download.pytorch.org/models/alexnet-owt-7be5be79.pth" to C:\Users\sissi/.cache\torch\hub\checkpoints\alexnet-owt-7be5be79.pth
100.0%
```

The alexnet model is split up into two components: *alexnet.features* and *alexnet.classifier*. The first neural network component, *alexnet.features*, is used to compute convolutional features, which are taken as input in *alexnet.classifier*.

The neural network alexnet.features expects an image tensor of shape Nx3x224x224 as input and it will output a tensor of shape Nx256x6x6 . (N = batch size).

Compute the AlexNet features for each of your training, validation, and test data. Here is an example code snippet showing how you can compute the AlexNet features for some images (your actual code might be different):

In [24]:
```
import os
classes = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I']

path = 'C:\\Users\\sissi\\OneDrive\\Desktop\\Code\\APS360\\APS360\\Lab 3\\Alex_feat

#alexnet.features: Huge stack of convolutional layers that looks at images and extr
def get_alex_feature(dataset, datatype):
    loader = torch.utils.data.DataLoader(dataset, batch_size=1, shuffle=True)
    n = 0
    folder = path + '/' + datatype + '/'

    for imgs, labels in iter(loader):
        features = alexnet.features(imgs)
        features_tensor = torch.from_numpy(features.detach().numpy())

        # Define exact folder path for specific letter/class
        label_name = str(classes[labels])
        save_dir = folder + label_name + '/'

        # Create the folder if it doesn't exist and save it into the folder
        if not os.path.exists(save_dir):
```

```
            os.makedirs(save_dir)
        torch.save(features_tensor.squeeze(0), save_dir + label_name + '_' + str(n)
        n += 1
```

Save the computed features. You will be using these features as input to your neural network in Part (b), and you do not want to re-compute the features every time. Instead, run *alexnet.features* once for each image, and save the result.

In [25]:
```
get_alex_feature(train_loader.dataset, 'train')
get_alex_feature(val_loader.dataset, 'val')
get_alex_feature(test_loader.dataset, 'test')
```

Part (b) - 3 pt

Build a convolutional neural network model that takes as input these AlexNet features, and makes a prediction. Your model should be a subclass of nn.Module.

Explain your choice of neural network architecture: how many layers did you choose? What types of layers did you use: fully-connected or convolutional? What about other decisions like pooling layers, activation functions, number of channels / hidden units in each layer?

Here is an example of how your model may be called:

In [ ]:
```
class AlexCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.name = "alex_cnn"

        # Convolution Layers
        self.conv1 = nn.Conv2d(256, 128, 3, padding=1) # Output: 128 x 6 x 6
        self.pool = nn.MaxPool2d(2, 2) # Output: 128 x 3 x 3

        # Fully connected layers
        self.fc1 = nn.Linear(1152, 128)  # 128 x 3 x 3 = 1152
        self.fc2 = nn.Linear(128, 9)   # Output = 9 for 9 classes

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = x.view(-1, 1152) # Flatten 3D tensor into a 1D line for the linear laye

        x = F.relu(self.fc1(x))
        x = self.fc2(x)

        return x
```

Layers: I chose to use 1 convolution layer and 2 fully connected layers. Since I am using a pretrained model, reducing the number of filters can shorten the training time but still have high accuracy. One pooling layer was used to shrink the tensor from 6x6 (Alex's output) to 3x3 so that the number of parameters needed in the fully connected layers can be reduced.

Number of Hidden Units & Channels: The convolution layer cuts down 256 incoming channels down to 128, resulting in 1152 parameters. Choosing 128 hidden units will efficiently compress 1152 raw input features down to 9 gestures while maximizing memory and proccessing efficiency.

Activation Function: I used ReLU to introduce non-linearity into the network which can prevent the math from collapsing into a single linear equation.

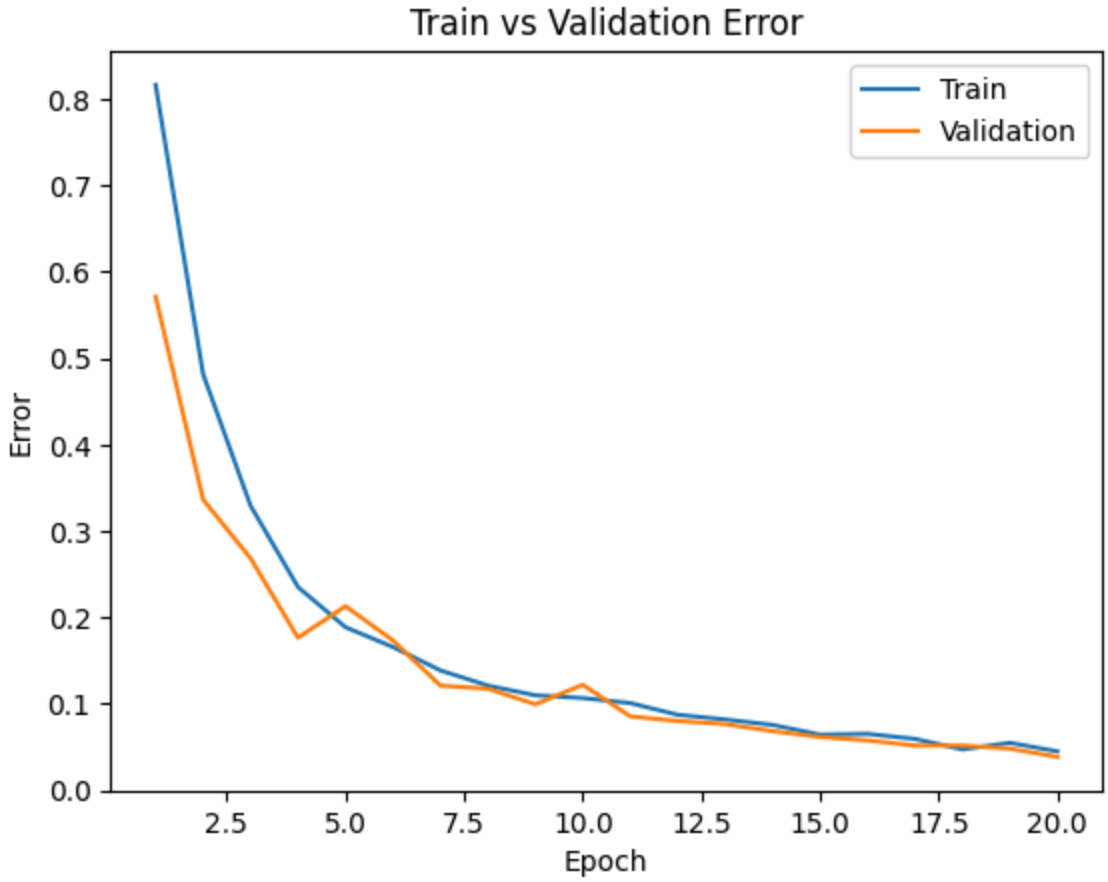
Part (c) - 5 pt

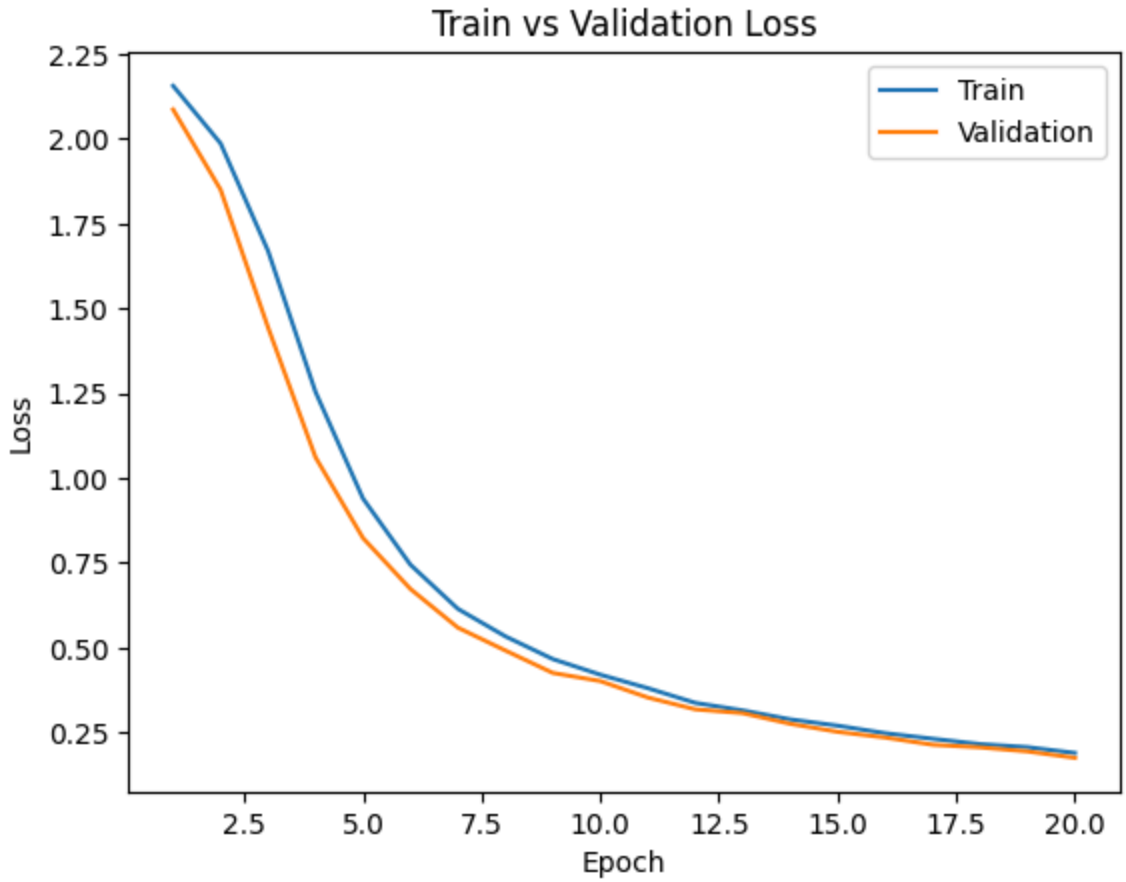
Train your new network, including any hyperparameter tuning. Plot and submit the training curve of your best model only.

Note: Depending on how you are caching (saving) your AlexNet features, PyTorch might still be tracking updates to the **AlexNet weights**, which we are not tuning. One workaround is to convert your AlexNet feature tensor into a numpy array, and then back into a PyTorch tensor.

In [40]:
```
Alex_cnn = AlexCNN()

train_path = 'C:\\Users\\sissi\\OneDrive\\Desktop\\Code\\APS360\\APS360\\Lab 3\\Ale
valid_path = 'C:\\Users\\sissi\\OneDrive\\Desktop\\Code\\APS360\\APS360\\Lab 3\\Ale
test_path = 'C:\\Users\\sissi\\OneDrive\\Desktop\\Code\\APS360\\APS360\\Lab 3\\Alex

# Create datasets
train_data_alex = torchvision.datasets.DatasetFolder(train_path, loader=torch.load,
valid_data_alex = torchvision.datasets.DatasetFolder(valid_path, loader=torch.load,
test_data_alex = torchvision.datasets.DatasetFolder(test_path, loader=torch.load, e

# Wrap data into data loaders
train_loader_alex = torch.utils.data.DataLoader(train_data_alex, batch_size=64, shu
valid_loader_alex = torch.utils.data.DataLoader(valid_data_alex, batch_size=64, shu
test_loader_alex = torch.utils.data.DataLoader(test_data_alex, batch_size=64, shuff

train_net(Alex_cnn, train_loader=train_loader_alex, val_loader=valid_loader_alex, b
alex_cnn_path = get_model_name(Alex_cnn.name, batch_size=64, learning_rate=0.001, e
plot_training_curve(alex_cnn_path)
```

```
Epoch 1: Train err: 0.8165840468679585, Train loss: 2.1563221863337927 |Validation e
rr: 0.5714285714285714, Validation loss: 2.08651362827846 |
Epoch 2: Train err: 0.48174853537629564, Train loss: 1.9865929569516863 |Validation
err: 0.33663812528165843, Validation loss: 1.850112874167306 |
Epoch 3: Train err: 0.32942767012167645, Train loss: 1.6696200711386544 |Validation
err: 0.26858945470932855, Validation loss: 1.44402939251491 |
Epoch 4: Train err: 0.2352410995944119, Train loss: 1.25388035433633 |Validation er
r: 0.17665615141955837, Validation loss: 1.0612823758806502 |
Epoch 5: Train err: 0.18882379450202794, Train loss: 0.9397744332041059 |Validation
err: 0.2131590806669671, Validation loss: 0.8232734135219029 |
Epoch 6: Train err: 0.1658404686795854, Train loss: 0.7444860867091587 |Validation e
rr: 0.17350157728706625, Validation loss: 0.6730119500841413 |
Epoch 7: Train err: 0.138801261829653, Train loss: 0.614371246950967 |Validation er
r: 0.12122577737719693, Validation loss: 0.5590346591813223 |
Epoch 8: Train err: 0.12122577737719693, Train loss: 0.533415128503527 |Validation e
rr: 0.11762054979720594, Validation loss: 0.4916264201913561 |
Epoch 9: Train err: 0.1099594411897251, Train loss: 0.46636911375182016 |Validation
err: 0.09959441189725102, Validation loss: 0.4255172908306122 |
Epoch 10: Train err: 0.106804867057233, Train loss: 0.42038536582674296 |Validation
err: 0.12212708427219468, Validation loss: 0.40148768084389824 |
Epoch 11: Train err: 0.10094637223974763, Train loss: 0.38066376873425073 |Validatio
n err: 0.08562415502478594, Validation loss: 0.3531155637332371 |
Epoch 12: Train err: 0.08742676881478144, Train loss: 0.3373169183731079 |Validation
err: 0.08021631365479946, Validation loss: 0.31822539227349417 |
Epoch 13: Train err: 0.08201892744479496, Train loss: 0.3150578230619431 |Validation
err: 0.07661108607480847, Validation loss: 0.3070289386170251 |
Epoch 14: Train err: 0.07570977917981073, Train loss: 0.28874012189252035 |Validatio
n err: 0.06849932401982875, Validation loss: 0.27564201099531993 |
Epoch 15: Train err: 0.0644434429923389, Train loss: 0.2700804067509515 |Validation
err: 0.061739522307345654, Validation loss: 0.25184097034590586 |
Epoch 16: Train err: 0.06534474988733664, Train loss: 0.24792833583695548 |Validatio
n err: 0.05768364127985579, Validation loss: 0.23510758323328837 |
Epoch 17: Train err: 0.059486255069851286, Train loss: 0.2315608867577144 |Validatio
n err: 0.051825146462370436, Validation loss: 0.2138720855116844 |
Epoch 18: Train err: 0.0473186119873817, Train loss: 0.21565818105425152 |Validation
err: 0.051825146462370436, Validation loss: 0.20614414640835355 |
Epoch 19: Train err: 0.05497972059486255, Train loss: 0.20673642030784062 |Validatio
n err: 0.04821991888237945, Validation loss: 0.19418881088495255 |
Epoch 20: Train err: 0.04506534474988734, Train loss: 0.1896165526338986 |Validation
err: 0.03875619648490311, Validation loss: 0.17524136496441706 |
Finished Training
Total time elapsed: 39.60 seconds
```


Train vs Validation Error
Train
Validation
Error
Epoch
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Train vs Validation Loss
Train
Validation
Loss
Epoch
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
2.25
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

Part (d) - 2 pt

Report the test accuracy of your best model. How does the test accuracy compare to Part 3(d) without transfer learning?

In [44]:
```
best_model = AlexCNN()
best_model_path = "Alex_training_data/model_{0}_bs{1}_lr{2}_epoch{3}".format("alex_
state = torch.load(best_model_path)
best_model.load_state_dict(state)

best_model.eval()

correct = 0
total = 0
with torch.no_grad():
    for imgs, labels in test_loader_alex:
        output = best_model(imgs)
        pred = output.max(1, keepdim=True)[1]
        correct += pred.eq(labels.view_as(pred)).sum().item()
        total += imgs.shape[0]

print('Final Test Accuracy: ', correct / total)
```

```
Final Test Accuracy:  0.9612438035150969
```

Final Parameters for Transfer learning:

batch_size: 64

learning_rate: 0.001

epoch: 20

In Part 3(d), the test accuracy without transfer learning was 79.58%, which is a lower accuracy compared to 96.12% achieved from AlexCNN. However, for AlexCNN, the best accuracy was actually achieved by lowering 30 epoch to 20. The reason for this parameter change is because when building the CNN from scratch, the first couple epoches are spent figuring out what basic shadows or curved lines looked like. With AlexCNN, the model gets a head start. It learns 9 gestures way faster and can finish training in 20 epoches instead. When I tried training AlexCNN using 30 epoches, it began to memorize the training set and overfitting, causing a lower test accuracy.

Additionally, a higher learning rate (0.0001 -> 0.001) was used since the data from AlexNet is more clean and organized, compared to all the noise from raw pixels. The optimizer can take slightly larger steps.